Flink as an Isotope Collector

Two propagation models now coexist in this project. They emit the identical wire format, so `05_isotope_view.fql` and every report is unchanged and cannot tell the collectors apart.

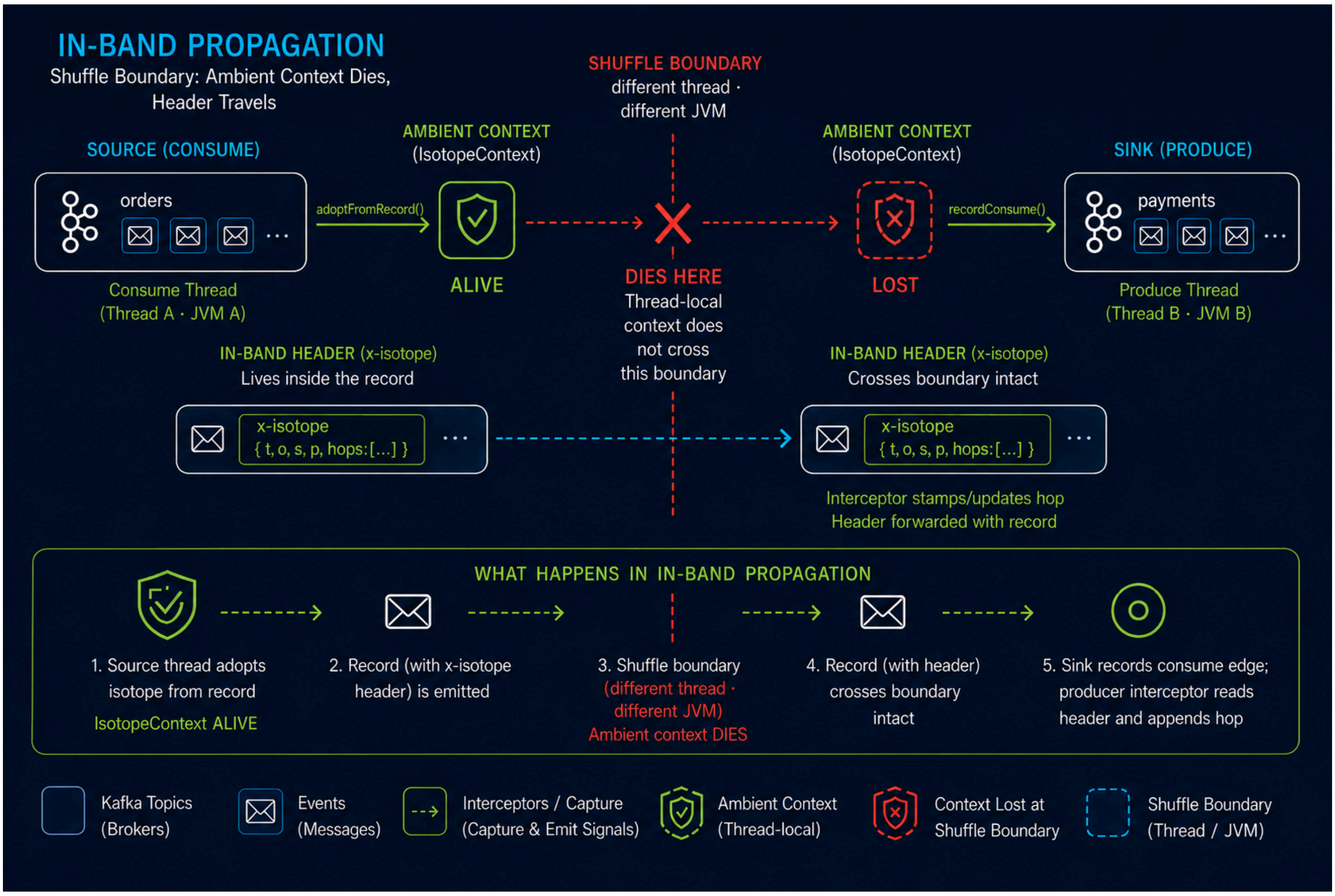
	Out-of-band propagation	**In-band propagation**
Where	Services in `app/`	Flink SQL (CP + CCAF)
Carrier	`IsotopeContext ThreadLocal`	The record's own Kafka headers
Stamped by	`IsotopeProducerInterceptor.onSend()`	`ISOTOPE_APPEND_HOP(...)` in `INSERT INTO`
Activation	`interceptor.classes` config	`CREATE FUNCTION` + a writable `headers` column

Table of Contents

1.0 Why a second model was needed

Out-of-band propagation is correct where it runs: one thread owns a whole consume→produce hop, so a `ThreadLocal` set at consume time is still there at `send()` time.

It cannot work in Flink. A shuffle serializes a record and resumes it on a different thread, usually in a different TaskManager JVM, so anything set *beside* the record is gone. Flink SQL is stricter still — there is no user-visible thread to attach to and no way to carry an opaque value past the planner, which is free to reorder, fuse, and eliminate rows.



The shuffle boundary is the whole argument. **Out-of-band**, the isotope lives *beside* the record and dies there; **in-band**, it lives *inside* the record and crosses intact.

> In Apache Flink, the term **shuffle (or data shuffling)** refers to the *process of redistributing data across different parallel partitions or network channels*. It is a critical operation used when data needs to be regrouped, combined, or re-organized across a distributed cluster—such as when performing a key-by grouping, a windowed aggregation, or a distributed join.

So in-band propagation moves the isotope *into* the record — which does not make the planner gentler, it makes the isotope something the planner is **obligated to respect**. `ISOTOPE_APPEND_HOP(...)` is an ordinary scalar expression over column values, and the planner may still reorder, fuse, push down, or split that plan across a shuffle. What it may not do is change the values the query computes. Carrying a column value is precisely the thing no rewrite is allowed to optimize away, so the isotope arrives at the sink intact.

The `ThreadLocal` had no such protection, for a precise reason: no expression in the query referenced it, so the planner saw no dependency and owed it nothing. The fix is not to work around the optimizer but to state the requirement in terms it already guarantees.

Two consequences follow, both intentional. The expression references `headers`, so projection pushdown must now keep that column alive from source to sink — a real dependency, visible to the optimizer. And Flink assumes a `ScalarFunction` is deterministic unless told otherwise, meaning it may constant-fold one, eliminate it as a common subexpression, or recompute it on failover; that is why the hop timestamp is a parameter rather than a clock read inside `eval()`. See §3.0 Constraints.

Note this is not a Flink limitation being worked around. Nothing in Flink core needs to change; the out-of-band *delivery mechanism* is what doesn't port, while the isotope format and semantics port intact.

2.0 What is wired, and where

Flink collects onto a **new parallel topic**, `orders.flink_enriched`. It reads `orders.placed` and re-emits each record with one extra hop. The three-stage service pipeline is untouched — `orders.{placed,enriched,fulfilled}` still belong entirely to out-of-band propagation — and Flink now appears as a producer in `topology_1m` and `bipartite_topology_1m`, where it was previously invisible.

Piece	CP (Minikube)	CCAF
UDF registration	`01_register_functions.fql`	`register_isotope_append_hop` in `setup-confluent-flink.tf`
Writable sink table	`07_flink_collector_sink.fql`	`flink_collector_sink` + two `ALTER`s in `setup-confluent-flink.tf`
Collector INSERT	`75_flink_collector.fql`	`insert_flink_collector` in `setup-confluent-flink.tf`
Teardown	`99_teardown.fql`	`terraform destroy`

`IsotopeReportsJob` registers `ISOTOPE_APPEND_HOP` from the classpath and runs the collector INSERT alongside the seven report INSERTs in the same `StatementSet`.

2.1 The writable headers column

The sink declares `headers` **without** `VIRTUAL`, which is what makes it writable — virtual metadata columns are read-only and excluded from the `INSERT INTO` target schema. On CCAF the equivalent is `ALTER TABLE ... MODIFY \`headers` MAP<STRING, STRING> METADATA`, since the four existing` ALTER`s add the column as` METADATA VIRTUAL`.

> Once persisted, the column is **mandatory in every `INSERT INTO`** that table. Only make it writable on tables Flink actually writes.

The source tables keep `MAP<STRING, BYTES>` (the Kafka connector's native metadata type) and the call site casts to `MAP<STRING, STRING>` on the way in. That keeps one UDF signature across both runtimes instead of forking the Java, and `MAP<STRING, STRING>` is the type Confluent documents for reading *and* writing headers.

The hop timestamp is passed **in** as `UNIX_TIMESTAMP() * 1000` rather than read inside `eval()` — a built-in the planner already knows not to fold, which keeps the non-determinism somewhere the optimizer can see it rather than hidden in the Java. See [§1.0 Why a second model was needed](#).

2.2 CCAF table shape

`DESCRIBE \`orders.placed`` against the live cluster returns:

Column	Type	Extras
`key`	`BYTES`	`BUCKET KEY`
`val`	`BYTES`	

Column	Type	Extras
`headers`	`MAP<STRING, BYTES>`	`METADATA FROM 'headers' VIRTUAL`

The Topic Catalog auto-imported it as the bytes-pair table rather than deriving columns from the Protobuf schema, so the CCAF sink mirrors `(key, val)` while the CP side declares a single `value` BYTES column locally. Each runtime keeps the shape its own catalog produces; only the header handling is shared.

2.3 Packaging the UDF

The UDF ships as the `ptf/` shadow JAR. One packaging detail is load-bearing, and getting it wrong fails on the first row rather than at registration.

`Isotope` declares `fromHeaders(org.apache.kafka.common.header.Headers)`. `IsotopeAppendHop` never calls it, but the JVM resolves that descriptor when linking the class, so `Headers` must be present at **runtime**, not merely at compile time.

`compileOnly` was tried first, on the assumption that the Flink Kafka connector would supply `kafka-clients`. It does not on CCAF. Probed against the live compute pool, `CREATE FUNCTION` succeeded and the statement planned and reached `RUNNING` — then died on the first row:

```
phase:   FAILED
detail:  UDF invocation error: exception raised in the user function code
         Message: org.apache.kafka.common.header.Headers
```

That failure timing is the trap: CCAF's function classloader does not expose `kafka-clients`, and nothing at registration time says so.

So `ptf/build.gradle` bundles `kafka-clients` (`transitive = false`, so no compression codecs) and relocates `org.apache.kafka` → `ai.signalroom.shaded.kafka`, which rewrites `Isotope`'s own reference in step and keeps the bundled copy from ever meeting the runtime's. It then ships **only** `common/header`, dropping the rest by package: 2 classes and ~1 KB, against 4,152 classes and 12 MB for the whole client.

`minimize()` cannot do that job — a type named only in a method descriptor is invisible to bytecode reachability analysis, so minimize strips `Headers` and the artifact fails exactly as `compileOnly` did. Only `Isotope` is ever loaded by the UDF; `IsotopeContext`, whose descriptors do reach `ConsumerRecord` and `Producer`, is never touched, and class loading is lazy, so its missing types cost nothing.

Confirmed in production on CCAF. With the relocated build deployed, querying the collector's output returns the stamped headers:

```sql
SELECT `headers`['x-isotope-this-service'], `headers`['x-isotope-hop-count']
FROM `orders.flink_enriched` LIMIT 3;
```

```
flink-enrich    2
flink-enrich    2
flink-enrich    2
```

`hop_count = 2` is the load-bearing part: hop 1 was the origin service producing to `orders.placed`, hop 2 is Flink. The count incremented rather than resetting, so the UDF rehydrated the inbound isotope and appended to it rather than minting a fresh trace — the same assertion `IsotopeAppendHopTest` makes, now observed end to end.

The durable fix remains a Kafka-free `isotope-format` artifact split out of `kafka-isotope`, which would make all of this unnecessary.

3.0 Constraints

3.1 1:1 Statements Only

1:1 statements only — tracing, not provenance. This is a deliberate boundary, not an unfinished feature, and the distinction is worth stating exactly.

An isotope records an *itinerary*: one identity and the ordered sequence of hops it visited — **a path**. Provenance records a *derivation*: for each output, the inputs that produced it — **a DAG**, and inherently many-to-one wherever data combines. The two coincide as long as every step is 1:1 — a DAG in which every node has exactly one parent is still a path — which is why `IsotopeAppendHop` and `IsotopeProducerInterceptor` both work.

A windowed aggregate breaks that 1:1 relationship. A `SUM` over 1,000 records has **1,000 parents**, and truthful provenance would need to identify all of them; the isotope format has no vocabulary for that. Therefore, **in-band propagation is restricted to 1:1 statements**, where a trace *is* a valid derivation record. Aggregations never emit a "representative" trace ID, because doing so would not merely produce incomplete provenance — **it would fabricate provenance by falsely implying that one input record produced the aggregate result.**

Note this is a property of the model, not of Flink: Kafka Streams and Spark face it identically. And in this pipeline it costs nothing, because the fan-in statements are all reports, and reports are terminal *as traced records*. `isotope_report_*_1m` is read by dashboards, never re-consumed as a business event that must carry a hop chain onward.

The trace-RCA report is the one case that reads a report rather than the event stream — `setup-ccaf-ai.tf` joins `isotope_report_stuck_trace_1m` to `ML_PREDICT` and emits an AI-written root cause. It is not a counterexample on either count: it is 1:1 (one alert in, one analysis out, via `LATERAL TABLE`), and it carries `trace_id` forward as a *column*, not as an isotope header, so it never re-enters the hop chain. Note it is CCAF-only and gated on `var.enable_trace_rca`; CP runs seven reports, CCAF seven or eight.

The 1:1 boundary happens to land exactly where tracing needs to reach.

Full provenance would **not** require a change to the isotope format. The fix is out-of-band rather than in-band: the merged output carries a **fresh trace**, while a side-channel *merge-edge* topic records the many-to-one edges — `(merge_trace_id, window_start, window_end, operator, contributing_trace_id)`, one row per contributing record — following the same architectural pattern `isotope_consume_edge_markers` already uses for consume edges. The wire format remains untouched, and `isotope_raw`, the typed views, and all seven reports remain exactly as they are.

Carrying that provenance **in-band** is what is impractical. An aggregation over 1,000 inputs would require 1,000 UUIDv7 trace IDs — **16 KB of raw IDs alone** against Kafka's typical 1 MB `max.message.bytes`, before encoding, hop history, headers, or the business payload. `Isotope` already acknowledges this boundedness through `MAX_HOPS` and the `truncated` flag: the format is intentionally not designed to carry unbounded provenance history along a path. A fresh trace at the merge is therefore not a workaround for that limitation — **it is the correct identity for the new record created by the merge**.

The load-bearing problem is **determinism, not vocabulary**. `Isotope.newTrace` currently mints its UUIDv7 using `ThreadLocalRandom`, which is harmless because the mint path in `IsotopeAppendHop` only fires for records that arrive untraced. At a merge, however, minting becomes the normal path. Because the merged record and its provenance edges would be emitted by two separate statements over the same window, independently generating their IDs could produce different `merge_trace_id` values under replay or re-evaluation, silently severing the relationship between them.

The merge ID therefore must be **derived rather than drawn**: a UUIDv7 whose timestamp field is the window end and whose random field is a truncated hash of `(pipeline, operator, window_start, window_end, group_key)`. Both statements can then independently compute the same ID; replay produces the same ID; and UUID ordering still follows event time. This extends the argument already made in §2.1 for passing the hop timestamp into `eval()` rather than reading the clock inside it: **anything that becomes part of trace identity must come from deterministic event data, not runtime state.**

That derivation requires only one change outside this repository. `Isotope`'s all-args constructor is package-private and only `newTrace(service, pipeline, originTsMs)` is public, so `kafka-isotope` would need an additive, non-breaking overload accepting a caller-supplied 16-byte trace ID, followed by a `ptf/build.gradle` dependency bump beyond `0.18.0`.

Three implementation decisions follow from constraints already documented here. First, the edge topic should use typed **Avro+Schema Registry**, like the report sinks, rather than the headers-only representation used by `isotope_consume_edge_markers`. Flink is producing the records, so a schema is available, and §3.2 rules out representing multiple contributing traces as repeated same-key headers. Second, the design should use **two INSERT statements**, rather than one PTF emitting both row types, because splitting tagged PTF output requires a `CREATE VIEW` over the PTF — a shape the CP Flink 2.1.2 Expander round-trip rejects, as documented in `60_stuck_trace_report.fql`. Third, the capability should be **opt-in on both runtimes**: `count = var.enable_merge_provenance ? 1 : 0` on CCAF, mirroring `var.enable_trace_rca`, and the currently unused `args` in `IsotopeReportsJob.main` on CP. When disabled, deployment remains byte-for-byte equivalent to today.

Two costs should be explicit. The edge stream emits one record per record *entering* the merge, so it roughly doubles that stage's write volume — an inherent cost of materializing a DAG edge list rather than a peculiarity of this design. And a trace walk still **terminates at the merge boundary**. The merge edges make ancestry queryable, not recursively traversable in Flink SQL, which lacks recursive CTEs. Closing over multiple generations of merges therefore belongs in a consumer or dashboard that walks the edge graph. In this design, **"full provenance" means that every derivation edge is recorded, not that a single Flink SQL query returns the complete ancestor set.**

> A recursive CTE (Common Table Expression) is a SQL subquery that repeatedly references its own results, allowing hierarchical or graph-like relationships to be traversed until the complete result set is reached.

None of this is implemented today, and the trigger for implementing it remains unchanged: it becomes worthwhile only when a **stateful Flink stage sits mid-pipeline** — such as an enrichment join or windowed deduplication whose output remains a business event that downstream services continue to trace. No such stage exists today: the collector is a projection, while every current fan-in operation produces a terminal report. Until that changes, **in-band propagation remains deliberately restricted to 1:1 transformations, where a single isotope can truthfully represent the derivation.**

3.2 Header Keys Must Be Unique

Confluent's ALTER TABLE reference states multi-key headers are unsupported. Isotope uses distinct keys throughout, but this rules out ever expressing hops as repeated same-key headers.

4.0 What this does not change

Out-of-band propagation is untouched. `IsotopeContext`, `IsotopeProducerInterceptor`, and the `interceptor.classes` wiring in `App.java` behave exactly as before, and remain the right model for the services.